Демонстрация из командной строки

Английский оригинал: demo_cli.md.

То же исследование, что в docs/demo.md, без окна. Это резервный вариант, если GUI не может работать на демонстрационной машине. Каждая команда и каждый вывод ниже выполнены на репозитории в том виде, как он закоммичен. Пути заданы относительно корня репозитория.

Настройте окружение один раз:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
export QT_QPA_PLATFORM=offscreen
```

Последняя строка держит Qt тихим, хотя ни одна из этих команд не открывает окно.

1. Полный конвейер: PCAP в проект в отчёт

```
python -m src.project.cli pipeline \
  --pcap tests/corpus/corpus_capture_01.pcapng \
  --project /tmp/demo.madrigal \
  --rule examples/corpus_rule_v2.json \
  --report /tmp/demo_report.md
```

Ожидается: одна строка JSON-сводки в stdout.

```
{"project": "/tmp/demo.madrigal", "normalized": ".../results/normalized.json",
 "sessions": 3, "streaming": false, "result": ".../results/result.json",
 "report": ".../reports/REPORT.md", "report_html": ".../reports/REPORT.html"}
```

Каталог проекта держит захват, нормализованный JSON, правило и отрисованный отчёт; /tmp/demo_report.md — тот же отчёт, записанный туда, куда попросили. Откройте reports/REPORT.html в браузере: он самодостаточен и не требует сети.

2. Применить правило v1 к одному направлению

```
python -m src.protocol.cli apply \
  --rule examples/corpus_rule_v1.json \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --out /tmp/result_v1.json
```

Ожидается сводка в stderr:

```
/tmp/result_v1.json: rule corpus_request v1, 60 matched, 0 mismatched
```

apply фреймит первую сессию, направление A_to_B. Записанный результат — документ, проходящий проверку по docs/schemas/result.schema.json; каждое сообщение несёт свои offset, length, раскодированные fields, status и provenance_range, привязывающий его к пакетам.

3. Проверить правило v1 по всему захвату

```
python -m src.protocol.cli verify \
  --rule examples/corpus_rule_v1.json \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --out /tmp/report_v1.json
```

Ожидается сводка в stderr:

```
rule corpus_request v1: 280 messages, 80 counterexamples
```

Отчёт считает каждое сообщение в каждом направлении по шести основным статусам:

```
"counts": {"matched": 60, "mismatched": 80, "not_applicable": 140, ...}
```

80 `mismatched` — это контрпримеры: запросы, чей байт команды `2` или `3`, что v1 запрещает. Каждый перечислен в `contradictions` с его сессией, направлением, смещением и байтами.

4. Применить правило v2 и перепроверить

```
python -m src.protocol.cli verify \
  --rule examples/corpus_rule_v2.json \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --out /tmp/report_v2.json
```

Ожидается сводка в stderr:

```
rule corpus_request v2: 280 messages, 0 counterexamples
```

счётчики: `matched=140`, `mismatched=0`, `not_applicable=140`. Уточнение расширило `command.expected` с `[1]` до `[1, 2, 3]`, и больше ничего не изменилось.

5. Сравнить две версии правила

```
python -m src.protocol.cli diff \
  --rule-a examples/corpus_rule_v1.json \
  --rule-b examples/corpus_rule_v2.json
```

Ожидается сводка (stderr), первые строки:

```
rule corpus_request: v1 -> v2
  ~ field command.expected: [1] -> [1, 2, 3]
```

Тело JSON отделяет diff правила (`changed_fields`) от любого diff отчёта, когда заданы `--report-a` и `--report-b`.

6. Скоррелировать запросы с журналом действий

```
python -m src.protocol.cli correlate \
  --rule examples/corpus_rule_v2.json \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --journal tests/corpus/corpus_journal.md \
  --out /tmp/correlation.json
```

Ожидается сводка внутри `/tmp/correlation.json`:

```
"summary": "60 of 60 messages correlated, 125 journal entries unmatched"
```

Против полного захвата (все направления) скрипт исследования сообщает то же согласие в большем объёме: 140 из 140 запросов на `corpus_capture_01` и 40 из 40 на `corpus_capture_02`, с раскодированной `target`, равной параметру журнала на каждом скоррелированном запросе.

7. Метрики качества

```
python -m src.protocol.cli metrics --report /tmp/report_v2.json
```

Ожидается сводка в stderr:

```
rule corpus_request v2: coverage 0.500, precision 1.000, counterexample density 0.0/100
```

Покрытие 0.5, потому что правило охватывает `A_to_B`, а половина сообщений — ответы. Точность 1.0, потому что ни одному matched-сообщению не противоречат.

8. Выгрузить правило как парсер

```
python -m src.protocol.cli export \
  --rule examples/corpus_rule_v2.json --format kaitai --out /tmp/parsers/
python -m src.protocol.cli export \
  --rule examples/corpus_rule_v2.json --format python --out /tmp/parsers/
```

Ожидается: два файла в `/tmp/parsers` (`corpus_request_v2.ksy` и `corpus_request_v2.py`), описание Kaitai Struct и автономный модуль Python, повторяющий фрейминг движка. Завершающий разделитель делает `--out` каталогом; без него путь берётся как выходной файл, и две выгрузки перезаписывают друг друга. Оба сгенерированы из правила, а не написаны вручную.

9. Пересобрать изложение исследования

```
python -m scripts.run_reference_investigation
```

Ожидается: `docs/REFERENCE_INVESTIGATION.md` переписан из захватов со своими разделами фрейминга, контрпримеров, уточнения, переноса, журнала и альтернатив. Команда падает, если корпус отсутствует, а не выдумывает данные.

10. Альтернативные чтения поля-гипотезы

```
python -m src.protocol.cli alternatives \
  --rule examples/corpus_rule_v2.json \
  --report /tmp/report_v2.json \
  --corpus tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --all
```

Ожидается сводка внутри JSON (по одному блоку на поле `hypothesis`):

```
"field_name": "command", "declared_meaning": "command", "best": "entropy_enum"
  entropy_enum   support 140  contradict 0  score 1.00
  low_cardinality support 140 contradict 0  score 1.00
  constant       support 60   contradict 80 score 0.43
```

`constant` — чтение, которое объявляло правило v1; на этом захвате большинство сообщений ему противоречат, а `entropy_enum` подходит ко всем. Это доказательство за уточнением, и оно произведено инструментом, а не заявлено. Команда печатает семь альтернатив для `command` и ставит `entropy_enum` первым; `--all` сообщает каждое поле-гипотезу сразу, а `--field command` ограничивает вывод одним полем.

11. Перенести проект и снова открыть

```
python -m src.project.cli export --project /tmp/demo.madrigal --out /tmp/demo.zip
python -m src.project.cli import --in /tmp/demo.zip --target /tmp/demo.moved
```

Возвращаемый JSON — `{"archive": "/tmp/demo.zip"}`; импорт возвращает `{"project": "/tmp/demo.moved", "broken": []}`, где пустой список `broken` значит, что каждый захват совпал со своей записанной контрольной суммой. Снова откройте перенесённый проект:

```
python -m src.project.cli open --path /tmp/demo.moved --show
```

Ожидается: манифест проекта, с захватом, зарегистрированным под `captures/corpus_capture_01.pcapng`, и его sha256. Несовпадение контрольной суммы прерывает импорт, а не импортирует изменённый файл.

Тайминг

шаг	минут
1 конвейер	2
2–4 применить и проверить	3
5 diff	1
6 корреляция	1
7 метрики	1
8 выгрузка	1
9 исследование	1
10 перенос проекта	1
итого	11